

윤경일 (Kyoungil, Yoon)

끊임없이 탐구하며, 성장하는 개발자



CONTACT

Github. <https://github.com/kidreamx>

Introduction

도전과 학습을 통해 정보를 파악하고 항상 소통하며 부족한 점을 배우고 정보를 공유하며 성장하는 개발자입니다.

SKILL



AI



- LangGraph를 통해서 AI-Agent 서비스를 개발할 수 있습니다.
- AI-API-Token을 Rag 기법을 활용하여 AI 챗봇을 만들 수 있습니다.
- 하이브리드 RAG · 온톨로지 기반 AI 서버를 구축할 수 있습니다.



Spring



- Spring MVC을 활용한 **Restful API 설계** 경험이 있습니다.
- Spring Security, JPA등 다양한 Spring Framework 활용 능력이 있으며, 공신 문서 및 자료 조사를 통한 적용을 빠르게 할 수 있습니다.
- Kafka를 적용하여 메시지 큐 클러스터 구축 및 비동기 파이프라인을 구현 할 수 있습니다.



Git



- Git-flow에 익숙하며, 브랜치 관리 및 코드 리뷰 등 협업에서의 규칙을 준수합니다.
- 여러 개발자와의 협업 중 발생한 충돌 현상을 Git을 통해 해결할 수 있습니다.
- Github 동기화 프로젝트를 통해서 merge 충돌 로직을 구현한 경험이 있음



JAVA



- 프로젝트에서 주력 언어로 사용하며, 활용 능력에 대하여 가장 자신있는 언어입니다.
- HttpServlet을 통해 웹 애플리케이션 개발 경험이 있습니다.
- 알고리즘 및 자료구조를 적용해서 시간복잡도와 공간복잡도를 개선 했던 경험이 있습니다.



Infra



- Jenkins를 활용한 CI/CD 파이프라인을 구축할 수 있으며, 프로세스에 대한 이해가 있습니다.
- 파라미터 빌드 방식으로 필요한 부분만 빌드 가능하게 파이프라인을 구축할 수 있습니다.
- 도커 및 도커 컴포즈 파일을 요구사항에 맞게 작성 가능합니다

Community



ETC



Education

삼성 청년 SW AI 아카데미 13기

(ssafy)

- 2025.01~2025.12
- 1 학기 프로젝트 최우수상 수상
- 2 학기 프로젝트 우수상 2 회

조선대학교

- 2019.03 ~ 2025.02
- 컴퓨터공학과 학사 졸업
- 종합평점 3.96/4.5
- 전공평점 4.05/4.5

Awards

2025.11 삼성청년SW아카데미 자율프로젝트 우수상

2025.08 삼성청년SW아카데미 공통프로젝트 우수상

2025.06 삼성청년SW아카데미 관통프로젝트 최우수상

2024.12 조선대학교 프로그래밍 경진대회 대상

2024.08 오아시스 해커톤 오아시스 특별상

Certificates

2025.03 삼성 상시 SW 역량테스트 Level A+

2025.08 OPIC IH(intermediate High)

2025.09 SQLD

2024.12 정보처리기사

Project Summary



제조 기업을 위한 AI 업무 워크스페이스(3p)

AI 기반의 커플 다이어리 및 일정 앱(9p)

시간,장소,상황에 따른 개인 맞춤 옷 추천 서비스(14p)

초보 개발자들을 위한 간편 코드 리뷰 플랫폼(18p)

관광지 추천 및 여행 일정 추천 플랫폼(25p)

전통시장 공동구매 플랫폼(29p)

조선대학교 교내 종합정보 앱

AXCORE WorkSpace

제조 기업을 위한 AI 업무 워크스페이스

2026.08.24~진행 중 (2인)

역할

Backend, AI, Infra

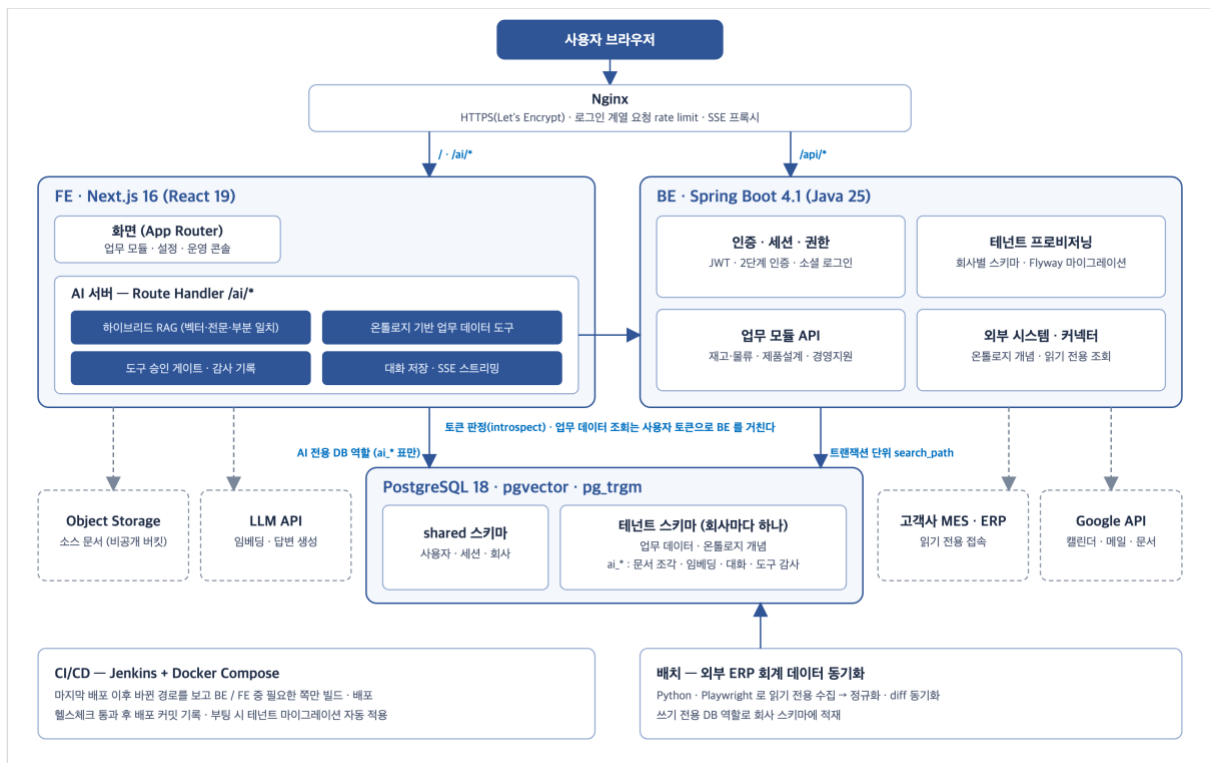
팀원 구성

Frontend 1 | **Backend · AI · Infra 1**

개발환경

Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4
Backend	Java 25, Spring Boot 4.1, Spring Security, JPA, Flyway
DataBase	PostgreSQL 18, pgvector, pg_trgm, Object Storage
AI	OpenAI API, AI SDK, RAG, 온톨로지
Infra	Jenkins, Nginx, Certbot, Docker, Docker-compose, Naver Cloud
Batch	Python, Playwright

Preview



시스템 아키텍처

담당 업무

하이브리드 RAG 기반 AI 대화 서버 구축

! 회사 문서를 근거로만 답하는 AI 서버를 색인부터 스트리밍까지 직접 설계 · 구현

- PDF · DOCX · XLSX · HWP 문서를 추출 · 분할 · 임베딩(pgvector)하는 **색인 파이프라인** 구축
- **벡터 · 전문 검색 · 부분 일치**를 RRF 로 합치고 LLM 재랭킹을 더한 **하이브리드 검색** 구현
- 재현율@8 · MRR · 거절 정확도를 재는 **평가 도구**로 검색 품질을 수치로 관리

온톨로지 기반 업무 데이터 조회 설계

! 데이터를 개념(동의어 · 속성 · 관계)으로 기술해 AI 가 사내 DB 와 고객사 MES · ERP 를 조회

- 조회마다 도구를 두면 100 개가 넘는 구조를 **창구 도구 1 개 + 온톨로지**로 설계해 도구 수 증가를 차단
- 외부 DB 구조를 읽어 **개념 초안을 자동 생성**하고, AI 제안을 운영자가 검토 · 저장하는 **온톨로지 스튜디오** 구현
- 시스템 간 키 값 겹침 · 고아 값 · 코드 체계 차이를 판정하는 **정합성 검사** 구현

AI 도구 실행 안전장치 설계

! AI 가 외부 서비스에 쓰기 작업을 할 때는 반드시 사람의 승인을 거치도록 설계

- 쓰기 도구는 **승인 게이트**를 거쳐서만 실행하고, 모든 도구 호출을 감사 테이블에 기록
- AI 서버를 **전용 DB 역할**(AI 테이블만 허용)로 접속시켜 침해 시 피해 범위를 제한
- 권한을 「목록에서 제외 + BE 재검증」 **두 겹**으로 적용해 권한 밖 데이터를 모델이 볼 수 없게 함

담당 업무

멀티테넌트 구조 및 테넌트 프로비저닝

! 회사 하나에 PostgreSQL 스키마 하나를 붙이는 스키마 분리형 멀티테넌시 구현

- 워크스페이스 개설 시 **스키마 생성과 Flyway 마이그레이션 전량을 한 트랜잭션**으로 묶어 반쪽 스키마가 남지 않게 함
- search_path 를 **트랜잭션 범위**로만 열어, 커넥션이 풀에 반납된 뒤 다른 회사 스키마가 남는 경로를 구조적으로 차단
- 부팅 시 기존 회사 스키마에 **마이그레이션 자동 적용**, 운영자 콘솔 API(개설 · 중지 · 해지 · 순회 배포) 구현

인증 · 보안 · 권한 시스템 개발

! Spring Security 기반 JWT 인증과 부서 · 직급 · 탭 단위 권한 모델 구현

- 이메일 · 소셜(Google · Naver) 로그인, 2 단계 인증, **refresh 토큰 회전과 재사용 감지**, 세션 즉시 폐기 구현
- 로그인 6 회 실패 잠금, 미확인 계정의 이메일 선점 차단 등 **계정 탈취 시나리오**를 기준으로 방어 로직 설계
- 「회사가 켜 기능 n 직급 권한 n 개인 부여」로 계산한 권한을 **화면 · API · AI 답변 범위**에 동일하게 적용

CI/CD 및 운영 환경 구축

! Jenkins + Docker Compose 기반 배포 파이프라인과 HTTPS 운영 환경 구축

- 마지막 배포 이후 **바뀐 파일 경로를 판정**해 BE / FE 중 필요한 쪽만 빌드 · 배포하는 파이프라인 구성
- Nginx + Certbot 으로 HTTPS 를 적용하고, 로그인 계열 요청을 **IP 당 분당 20 건**으로 제한

Troubleshooting

1. 근거 없는 질문에 답을 지어내던 RAG 환각 차단

🔗 관련 문서가 없는데도 AI 가 답을 만들어 내는 문제 발생

상위 8개 조각을 무조건 문맥에 넣어 무관한 조각이 항상 들어갔고, 「근거가 없으면 모른다고 답하라」는 분기가 실행되지 않았음

✓ 실제 문서로 유사도 분포를 재서 근거 문턱을 정하고 적용

- 문서 4건 · 질문 10개를 실측해 근거 있는 질문 최저 0.327 / 없는 질문 최고 0.282 로 갈리는 것을 확인하고 문턱을 0.30 으로 설정
- 문턱 판정을 재랭킹보다 먼저 하도록 순서를 바꿔 부작용을 해결 → 실측 질문 10개 모두 올바르게 분류

2. 품번을 놓치던 검색을 하이브리드 검색으로 개선

🔗 품번 · 전표번호처럼 글자가 정확히 맞아야 하는 질문에서 문서를 찾지 못함

벡터 결과가 한 건이라도 있으면 전문 검색을 건너뛰었고, 조사가 붙은 낱말은 다른 토큰이 되어 AND 검색에서 탈락했음

✓ 벡터 · 전문 검색 · 부분 일치를 합산하는 하이브리드 검색으로 재설계

- 세 검색의 순위를 $RRF(k=60)$ 로 합치고 질의를 AND → OR 로 완화, 품번 일부는 pg_trgm 부분 일치로 보완
- 상위 24개 후보를 LLM 이 재랭킹해 8개만 남기고, 평가 도구(재현율@8 · MRR)로 효과를 검증

3. 합계 질문에 답 없이 끝나던 AI 응답 문제 해결

🔗 「8월 손익 요약해 줘」 같은 질문에 AI 가 답을 내지 못하고 종료됨

모델이 전표 행을 직접 더하려다 도구를 20번 호출했고, 턴당 도구 단계 한도(6)에 걸려 답변 없이 끝났음

✓ 합계를 모델이 아니라 DB 가 계산하도록 구조 변경

- 도구에 집계 입력(group · sum)을 추가해 원천 DB 에서 GROUP BY 로 합계를 계산 (값은 바인딩)
- 마지막 단계에서는 도구 호출을 막고 글로 답하게 해 답 없이 끝나는 턴을 제거

Troubleshooting

4. 부하 테스트로 찾은 로그인 병목 해소

🔗 1차 부하 테스트에서 2 vCPU 서버가 동시 사용자 약 130명에서 붕괴

로그인이 DB 커넥션을 잡은 채 bcrypt(약 55ms)를 실행해 풀이 고갈됐고, 폭주 시 bcrypt 가 CPU 를 독점해 다른 요청 까지 지연됐음

✅ 비밀번호 대조를 트랜잭션 밖으로 분리하고 동시 실행 수를 제한

- @Transactional 을 걷어 bcrypt 를 커넥션 없이 실행하고 세션 발급만 짧은 트랜잭션으로 분리 (Hikari 풀 10 → 25, 대기 30초 → 3초)
- 세마포어(허가 4) 로 동시 대조 수를 묶어 초과 시 503 으로 빠르게 응답, Nginx 에서 IP 당 분당 20건으로 제한

5. AI 도구 승인의 동시 요청 중복 실행 차단

🔗 같은 승인이 동시에 들어오면 도구가 두 번 실행될 수 있음을 발견

승인 상태를 SELECT 로 확인한 뒤 UPDATE 하는 구조라, 두 요청이 모두 「대기 중」을 읽고 각각 실행할 수 있었음

✅ 조회와 변경을 한 문장으로 합쳐 승인을 선점

- 「UPDATE ... WHERE status = 'proposed' RETURNING」 한 문장으로 선점 → 동시 3요청 테스트에서 1건만 실행됨을 확인
- 실행 입력은 클라이언트 값이 아닌 서버가 기록해 둔 입력을 사용해 승인 내용 위조를 차단

6. 탭을 여러 개 쓰면 전체 로그아웃되던 문제 해결

🔗 정상 사용자가 갑자기 모든 기기에서 로그아웃되는 현상 발생

회전된 refresh 토큰이 다시 오면 무조건 탈취로 보고 전체 세션을 폐기해, 두 탭이 동시에 재발급을 요청해도 로그아웃됐음

✅ 회전 직후 30초 유예 창을 두어 정상 경합과 탈취를 구분

- 회전 후 30초 안에 들어온 옛 토큰은 access 토큰만 재발급하고 쿠키는 다시 심지 않음
- 유예 밖 재사용은 그대로 전체 세션 폐기 → 보안 수준은 유지하면서 오탐 로그아웃을 제거



ToGather

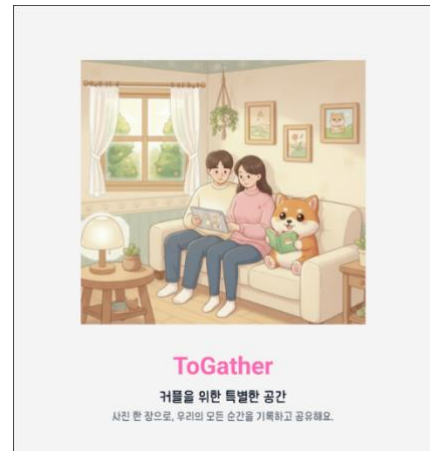
커플들을 위한 일상 다이어리 기록 앱

2025.10.10~2025.11.20 (6주/ 6인)

참고 링크



<https://github.com/SSAFY-ToGather/ToGather>



역할

Backend(50%), Infra(50%)

팀원 구성

Frontend(2) | **Backend 4** | **Infra 1**

특이사항

SSAFY 프로젝트 우수상

개발환경

Frontend	React Native, Expo, TypeScript ,Expo Router, Zustand, TanStack Query
Backend	Java17, Spring Boot, JPA, SSE, Kafka, FastAPI, Firebase Cloud Messaging
DataBase	MySql, Qdrant,
AI	GPT-4o API, Gemini API
Infra	Jenkins, Nginx, Docker, Docker-compose, Amazon S3, Ubuntu EC2

Preview – (1)



커플 추억 기반 AI 챗봇



AI 사진 자동 요약

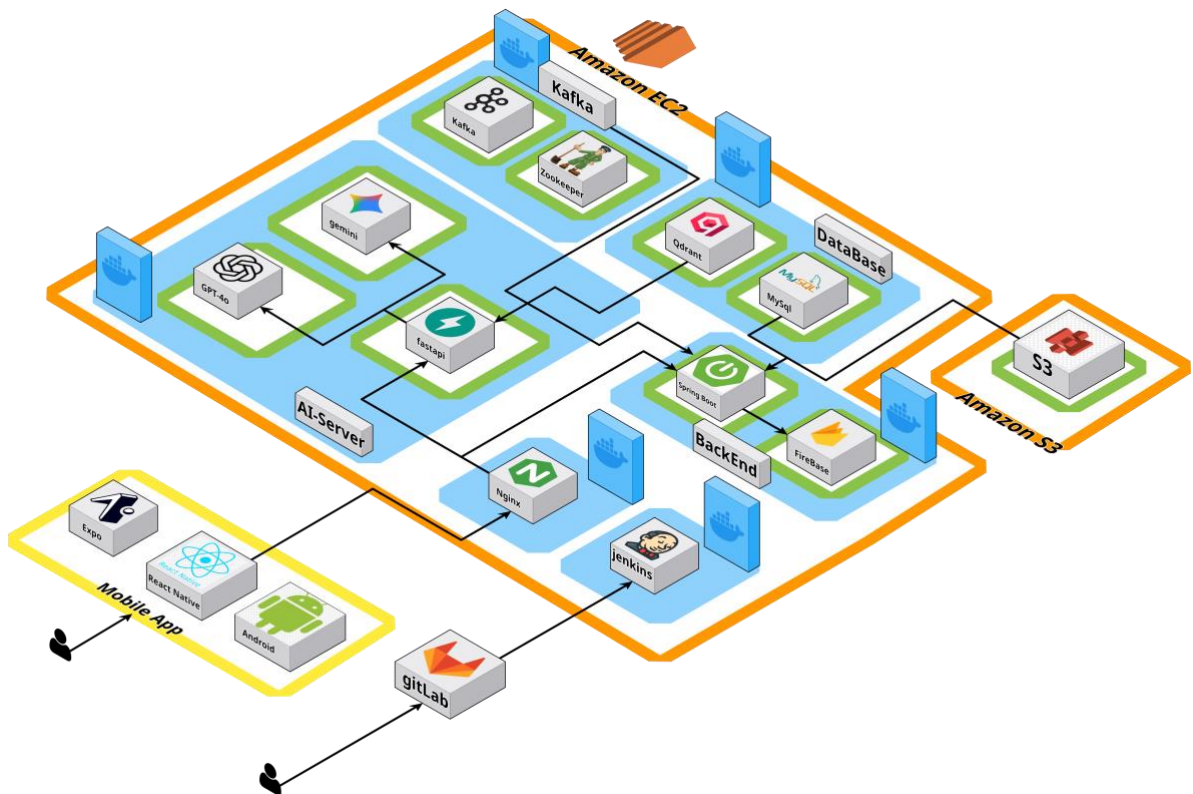


키워드 기반 장소 검색



일정 저장 및 알림 서비스

Preview – (2)



시스템 아키텍처

담당 업무

CI/CD 파이프라인 설계 및 구축

! Jenkins 파이프라인을 통해 CI/CD 파이프라인 구축

- 개발 특성상 개별 빌드의 필요성 때문에 **파라미터 빌드** 방식을 채택해 상황별로 필요한 **container** 만 빌드하게 설계
- **Docker Compose** 를 이용한 **DB/Backend/AI/Kafka/nginx** 컨테이너 환경 통합 관리

Kafka와 Zookeeper를 활용한 분산 이벤트 처리 환경 구성

! 대용량 트래픽 처리를 위해 메시지 큐(Kafka) 클러스터 구축 및 비동기 파이프라인을 적용

- 대용량 트래픽의 안정적인 처리를 위해 메시지 큐(Kafka) 클러스터를 구축 및 **비동기 파이프라인**을 적용하여 **시스템 응답 속도 및 처리 효율성 개선**
- Kafka + SSE 연동을 통해 AI 서버와의 통신 시간 **95% 단축 (10 초 → 0.5 초)**
- 애플리케이션 시작 시 파티션 자동 조정 기능 및 SSE 연결 해제 감지 구현을 통해 리소스 자동 최적화 시스템 구축

커플 추억 다이어리 서비스

! SpringBoot를 활용한 Post , Section 계층 구조 기반 다이어리 시스템 개발

- Post-Section 계층 구조 설계 및 Section CRUD 시 Qdrant 벡터 DB에 자동 임베딩하여 RAG 챗봇이 과거 다이어리를 검색 가능하도록 연동
- Cursor 기반 페이징(Slice)을 적용하여 Offset 방식 대비 대용량 데이터에서도 일정한 조회 성능을 유지하고, COUNT 쿼리 제거로 응답 속도 개선
- @Query + LEFT JOIN FETCH를 활용하여 Post-Section 조회 시 N+1 문제를 해결하고, 100 개 게시글 조회 시 101번 쿼리를 1번으로 감소시켜 약 100배 성능 향상 달성

Troubleshooting

1. AI 이미지 분석 속도 20배 향상

🔗 어떤 문제가 있었는가?

기존 동기 방식의 이미지 분석 로직에서 100장의 이미지를 처리하는 데 20분이 소요되고, 다수 사용자 접속 시 병목 현상이 발생하는 문제를 확인했습니다

✅ Kafka 와 Zookeeper를 활용해 분산 처리 시스템 구축

- Kafka와 Zookeeper를 도입하여 비동기 이벤트 기반 아키텍처로 재설계했습니다. 데이터 처리량을 높이기 위해 Topic 파티션을 20개로 확장하고, 이에 맞춰 Consumer 스레드 또한 20개로 증설하여 병렬 처리 환경을 구성했습니다
- 이를 통해 작업 처리 속도를 기존 대비 20 배 향상시켰으며, 10 명의 동시 사용자 요청에 대해서도 10 배 빠른 응답 속도를 확보하여 서비스 안정성을 높였습니다.

2. 상황에 따른 파라미터 빌드를 통한 개발 효율성 향상

🔗 Dev 수정 후 머지할때 서버가 멈추는 문제 발생

기존 빌드 방식에서는 빌드시 모든 도커 container가 재시작하는 파이프라인으로 인해 FrontEnd와 BackEnd 연동 작업의 불편함을 발생시킴

✅ 개발 파트별로 따로 빌드를 진행할 수 있는 파라미터 빌드 파이프라인을 구축하여 해결

- AI , BackEnd(SpringBoot), DB을 따로 빌드할 있게 설정하기 위해 파라미터로 설정하였고 이를 통해 작업 후에 해당 파트만 따로 빌드를 진행하면서 문제점을 해결함.
- 개발 후 프론트엔드 팀원으로부터 분리 설계가 가능해서 쾌적하게 개발할 수 있었다는 평가를 받게됨

3. 인덱스 적용을 통해 조회 속도 향상

🔗 테스트로 많은 데이터를 조회시 속도가 느려지는 것을 발견함

부하 테스트를 진행하던 때에 post와 section 조회시 속도가 느려짐을 느끼게 되어 향상시키기 위해 방법을 찾아봄

✅ Post테이블의 복합 인덱스(post_id, post_date, couple_id)를 적용해 속도 향상

- 100만건의 데이터를 기준으로 테스트 하였을 당시에 1200ms에서 890ms로 약25%의 성능 향상을 이룸.



OQ

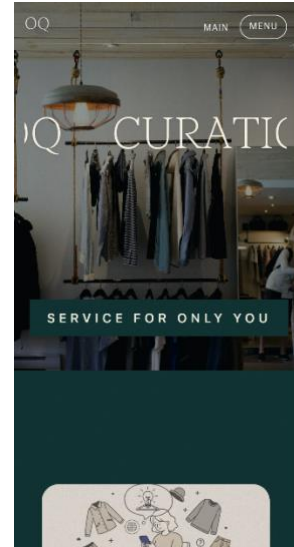
시간,장소,상황에 따른 옷 추천 도우미 앱

2025.08.25~2025.10.02 (5주/ 6인)



참고 링크

<https://github.com/YesHaddop/OQ>



역할

AI(70%), BackEnd(30%)

특이사항

SSAFY 베스트 멤버 수상

팀원 구성

Frontend(2) | **Backend** 4 | Infra 1 | **AI** 1 | Big Data 1

개발환경

Frontend React, Tailwind

Backend Java17, Spring Boot, JPA, SSE, Kafka, FastAPI

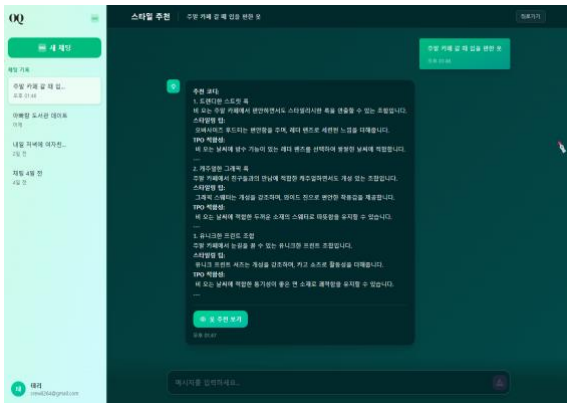
DataBase MySql, Redis

AI GPT-4o-mini API, LangGraph

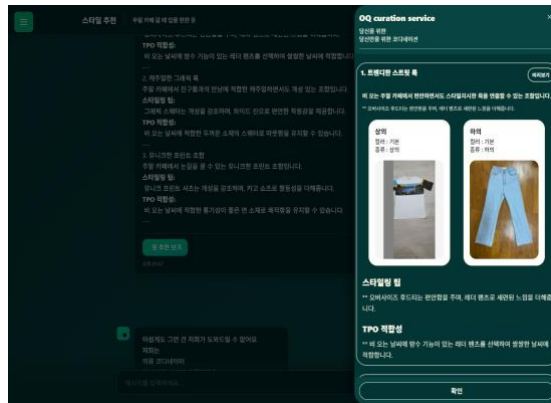
Infra Jenkins, Nginx, Docker, Docker-compose, Amazon S3, Ubuntu EC2

Big Data Hadoop, Apache Spark

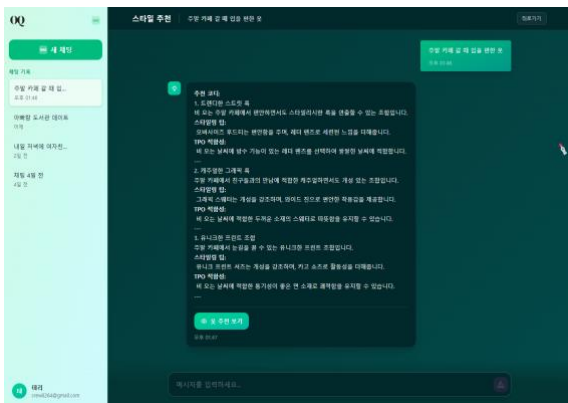
Preview



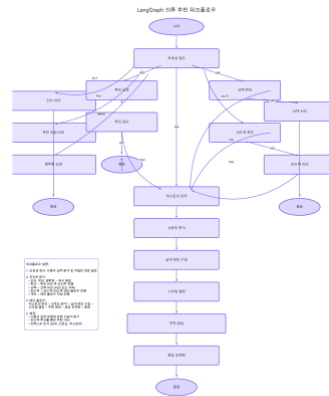
AI 채팅 페이지



최종 옷 추천 페이지



메인 페이지



AI 옷 추천 워크플로우(pipeline)

담당 업무

LangGraph 기반 AI 에이전트 및 TPO 추천 로직 설계

! 멀티턴 대화의 문맥 유지와 외부 데이터 연동을 통해 고도화된 상황 맞춤형 추천 서비스 구현

- **LangGraph 기반 상태 머신 설계:** 복잡한 대화 흐름을 상태 머신으로 관리하여 멀티턴 대화의 컨텍스트를 안정적으로 유지하고, OpenAI GPT 모델을 활용한 정교한 의류 추천 로직 구현.
- **TPO(Time, Place, Occasion) 분석 엔진:** 실시간 기상 API 및 하둡 빅데이터 API를 연동하여 사용자의 현재 상황과 트렌드를 반영한 **하이브리드 추천 알고리즘** 개발.
- **사용자 데이터 통합 관리:** MySQL 기반의 대화 히스토리 및 사용자 선호도 데이터베이스 설계로 개인화된 추천 경험 제공 및 데이터 일관성 확보.

비동기 파이프라인 및 SSE 적용을 통한 성능 최적화

! 기구축된 Kafka를 활용한 비동기 통신 구조 설계 및 실시간 스트리밍 도입으로 응답 속도 개선

- **시스템 응답 속도 개선:** Spring Boot 백엔드와 AI 서버 간 통신을 위해 구축된 Kafka 비동기 파이프라인에 SSE를 적용하여 **시스템 응답 속도 33% 단축 (10초 → 3초)**.
- **실시간 스트리밍(SSE) 연동:** AI 모델의 생성형 답변을 실시간으로 전달하기 위해 SSE(Server-Sent Events) 기반 응답 기능을 개발하여 사용자 체감 대기 시간 대폭 감소.
- **병렬 처리 및 리소스 최적화:** ThreadPoolExecutor를 활용해 최대 10개의 요청을 병렬 처리하고, correlationId를 통해 요청-응답 매칭의 정확성 확보.

서비스 백엔드 구조 설계

- 백엔드 구조 설계 : DataBase 구조 설계 및 **SpringBoot**를 통해 **회원 API** 설계

Troubleshooting

서비스 목적에 최적화된 에이전트 아키텍처 선정 및 설계

🔗 AI-Agent를 개발할 당시에 알맞은 에이전트 아키텍처 선정에 있어서 고민이 있었음

✅ 복잡한 로직 설계와 실시간 처리를 결합한 LangGraph를 선정

- StateGraph 기능과 Conditional Edge 기능을 통해서 복잡한 상태 관리가 쉽고 답변 정확도를 올려주고 실시간 응답이 가능한 장점으로 인해서 LangGraph를 선정하게 됨
- 총 13 가지의 Conditional_Edge 와 총 22 가지의 상태필드를 통해서 예상되는 LangChain 기술보다 코드 복잡도가 25% 감소하면서 유지 보수성을 확보함 또한 컨텍스트가 소비하는 토큰 소비량도 약 15% 정도 절감됨

AI-Agent 정확도 35% 향상 및 응답속도 60% 향상

🔗 1. AI-Agent와 대화 시 질문 해석과 답변 응답이 정확하지 않음을 발견함

✅ 텍스트 파싱 방식에서 Json 매핑 방식 및 실시간 날씨 API를 통해 자동으로 시간정보 입력

- 1대1 매핑 방식인 텍스트 파싱 방식에서 카테고리별 JSON 매핑 방식을 채택하면서 정확도가 60%에서 95%로 35% 향상함
- TPO 정보중 시간정보는 실시간 날씨 API를 통해서 오늘, 내일, 모레 와 같은 특정 시간 정보가 주어지면 자동으로 날씨 정보도 포함해 답변하도록 해 평균 재추천 횟수를 3회 -> 1.5회로 줄임

🔗 2. AI-Agent 응답 속도가 느리고 처리 단계를 알 수 없는 문제를 발견함

✅ 응답 처리 방식을 동기 방식에서 실시간 스트리밍 (SSE) 방식 및 AI 모델을 교체하며 60% 속도 향상및 체감 속도 개선

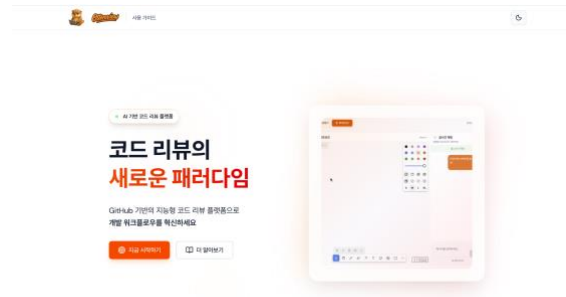
- 기존 5초의 블로킹 대기 시간을 0.1초 내 응답 시작의 실시간 스트리밍 방식으로 개선하여 사용자 체감 속도를 50배 향상
- 초기에 GPT-4-Turbo 모델을 사용하면서 시간과 토큰이 많이 소모 된다는 사실을 알게 됨. 이를 해결하고자 토큰 비용도 저렴하고 에이전트 반복추론에 능한 GPT-4o-mini로 교체하면서 5초->2초로 60% 응답 속도 향상



Otterreview

AI 기반 협업형 코드 리뷰 및 충돌 해결 플랫폼

2025.07.07~2025.08.21 (7주/ 6인)



참고 링크

<https://github.com/kidreamx/otterreview>

역할

Backend(50%), Infra(50%)

특이사항

SSAFY 프로젝트 우수상

팀원 구성

Frontend(2) | **Backend** 4 | **Infra** 1 | AI 1

개발환경

Frontend React , TailWind, Zustand

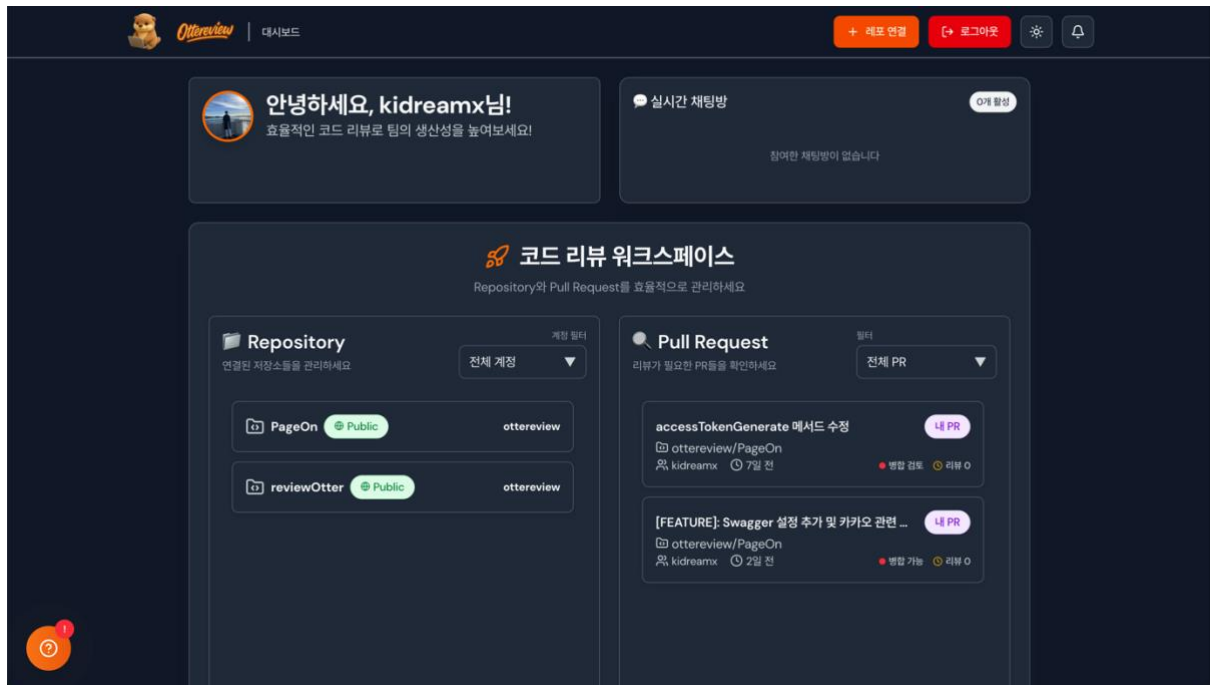
Backend Java17, Spring Boot, Github Webhock, FastAPI,

DataBase MySql, Redis, PineCone

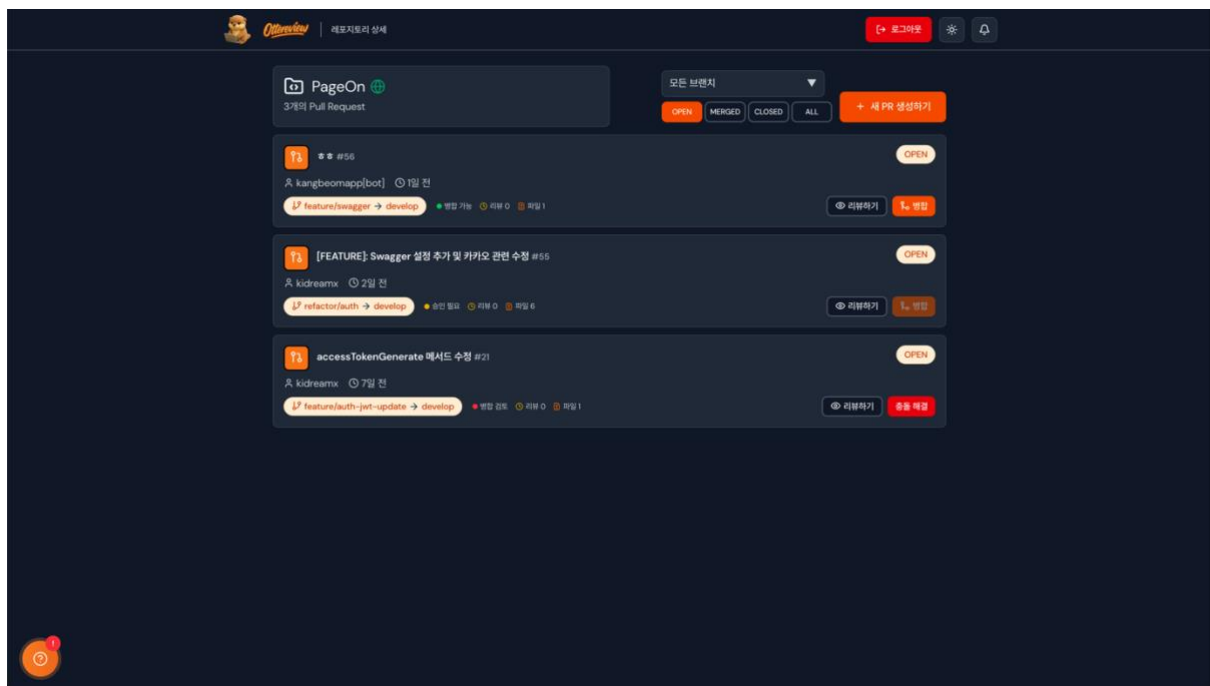
AI GPT-4o API, Gemini API, Langchain,

Infra Jenkins, Nginx, Docker, Docker-compose, Amazon S3, Ubuntu EC2

Preview – (1)



메인 페이지(다크모드)



레포지토리 상세

Preview – (2)

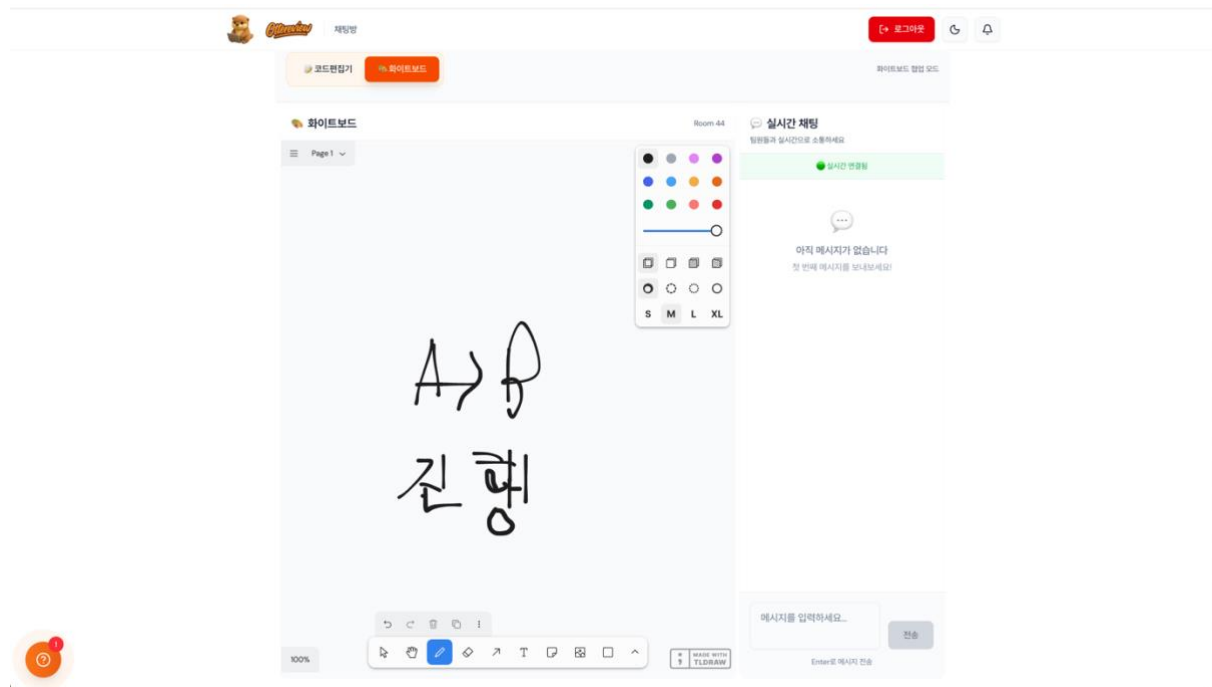


PR 상세조회 –(1)

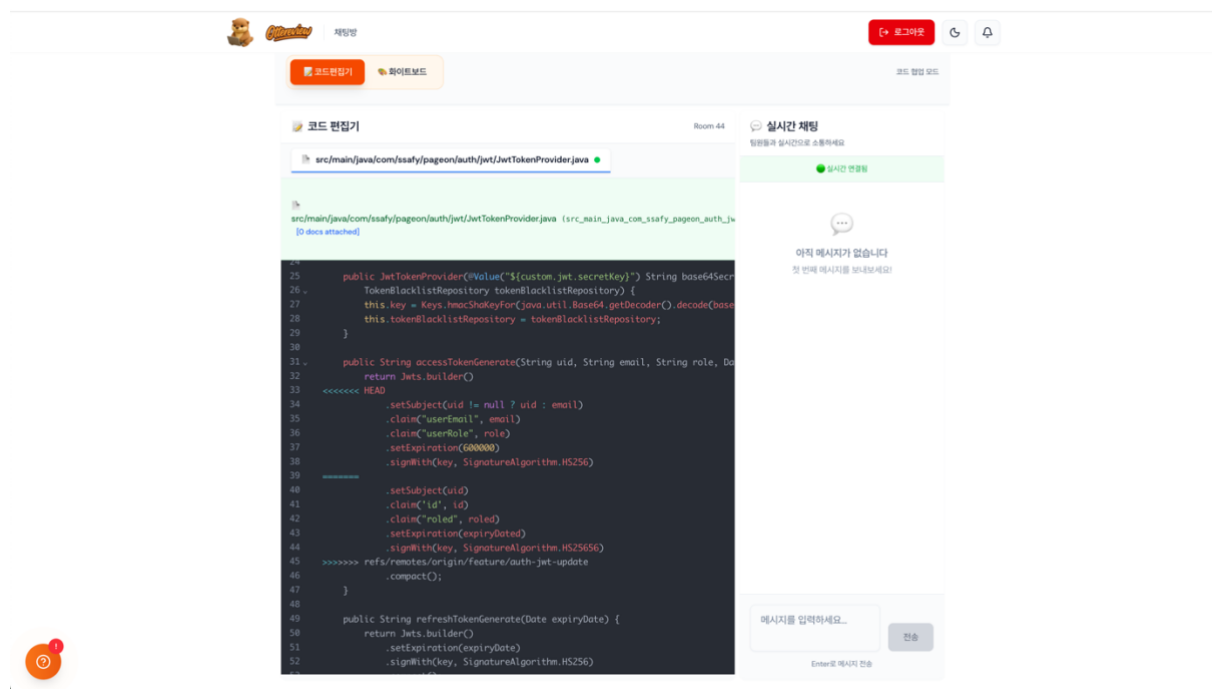


PR 상세조회 –(2)

Preview – (3)

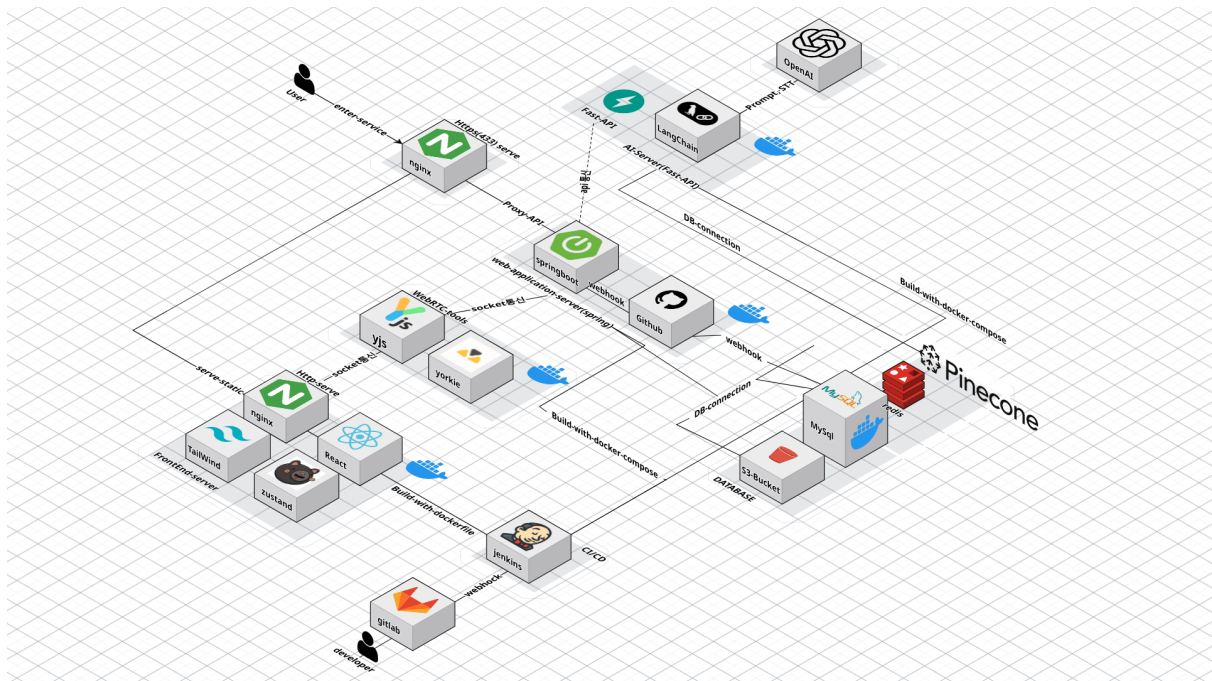


화이트 보드 기능



공동 코드 편집 기능

Preview – (4)



시스템 아키텍처

담당 업무

Github Webhook 적용 및 Github App Installation 적용

! 실시간 이벤트 기반 데이터 동기화 및 설정 복잡도를 낮춤

- 10종 Webhook 이벤트 처리를 실시간으로 수신하여 프로젝트 상태 자동 갱신
- SSE를 적용하여 실시간으로 사용자가 변경사항에 대한 알림을 받을 수 있게 적용
- Installation Callback 시점에 조직/개인 저장소, 브랜치, PR 정보를 일괄 동기화하는 자동화 파이프라인 구축.

프로젝트 배포 환경 구축 및 구현

! EC2, Jenkins ,Nginx , docker-compose를 활용하여 CI/CD 환경 구축

- Jenkins items중 빠른 환경 구축을 위해 Free-Style을 선택해 Jenkins의 초보 친화적인 UI를 활용하고 빌드 시간 단축을 위해 CI시에는 Docker hub에 올리고 CD할때에는 다운받는 파이프라인으로 구축함
- 배포환경에 모든 프로그램을 Docker Container로 올려 독립성과 경량화에 신경썼고 부하 분산 및 중앙집중화를 위해 Https를 처리하는 Nginx와 Ec2 서버내 정적파일 서빙을 위한 Http 통신용 Nginx를 설치하여 설계함

Repository 및 branch 연동 및 API 기능 구현

! SpringBoot 와 Github Webhook를 활용하여 실시간 연동 및 API 기능 구현

- MVC 패턴 적용 및 Service 단 SingleTon 디자인 패턴 적용을 통해 인스턴스를 효율적으로 관리하도록 하는 Repository 및 Branch 도메인 API 구현함
- hub4j Github API보다 Github Webhook이 32% 빠르게 동기화 가능하다는 팀원들에게 전달 후 Github Webhook과 Github installation Token을 적용함 또한 처음과 업데이트시의 로직을 분리 설계해 유지 보수성을 높임

Troubleshooting

GitHub Webhook 이벤트 설계로 동기화 속도 75% 개선

🔗 Github API를 사용할 시 저장소 1개당 평균 2초의 동기화 시간이 소요됨

✅ API 호출 부하를 줄이고 데이터 정합성을 확보한 이벤트 기반 아키텍처 구현

- 이벤트 기반 전환: GitHub Webhook을 도입하여 변경 사항 발생 시 GitHub에서 서버로 즉각 신호를 보내는 수동적 수신 구조로 변경.
- 데이터 정제 최적화: Webhook으로 전달받은 페이로드를 도메인별로 분리하여 필요한 데이터만 선별적으로 가공 및 DB에 저장하는 최적화 로직 설계.
- 호출 효율화: 불필요한 전체 조회 API 호출을 제거하고, 변경된 데이터만 부분 업데이트하여 네트워크 및 DB 부하 감소.

프로젝트 관리 방법론 개선을 통한 팀 생산성 2.5배 향상

🔗 폭포수 모델로 초반에 진행하며 주간 진행도가 10%에 그치는 문제 발생

✅ 폭포수 모델에서 애자일로의 전환을 통한 개발 가속화 및

협업 효율 극대화

- 애자일 방법론을 도입하여 변화하는 요구사항에 유연하게 대응하기 위해 매주 단위로 목표를 설정하고 검토하는 스프린트 체계로 전환함
- 방식 도입 직후, 주간 진행도가 기존 10%에서 25%로 약 2.5배 향상되는 결과를 얻음
- 매주 정기적인 스프린트 회의를 통해 JIRA에 스토리에 작업을 부여하여 각 파트별 진행 상황을 투명하게 공유하고, 발생 가능한 이슈를 조기에 발견하여 즉각적으로 수정하는 피드백 루프 구축함.



저기어때

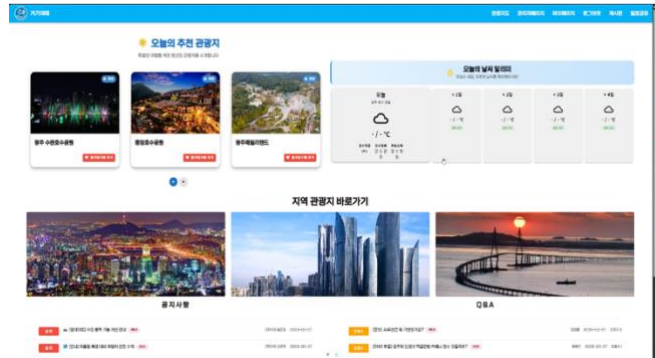
관광지 추천 및 여행 일정 추천 플랫폼

2025.05.06~2025.05.30 (4주/ 2인)

참고 링크



https://lab.ssafy.com/jun000405/EnjoyTrip_FINAL_PJT



역할

Backend(50%), FrontEnd(50%)

특이사항

SSAFY 1학기 최종 프로젝트 최우수상

팀원 구성

Frontend 2 | **Backend** 2 | **Infra** 1

개발환경

Frontend Vue.js , Vuetify,

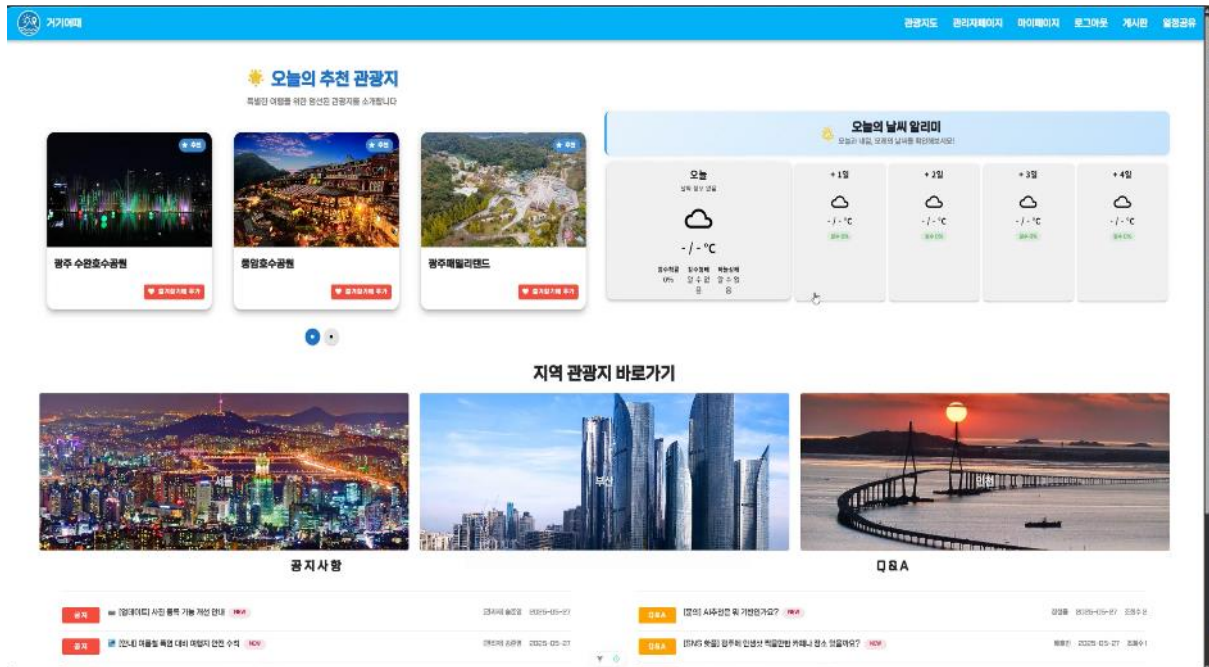
Backend Java17, Spring Boot , MyBatis , JWT , Spring Security

DataBase MySql, Redis

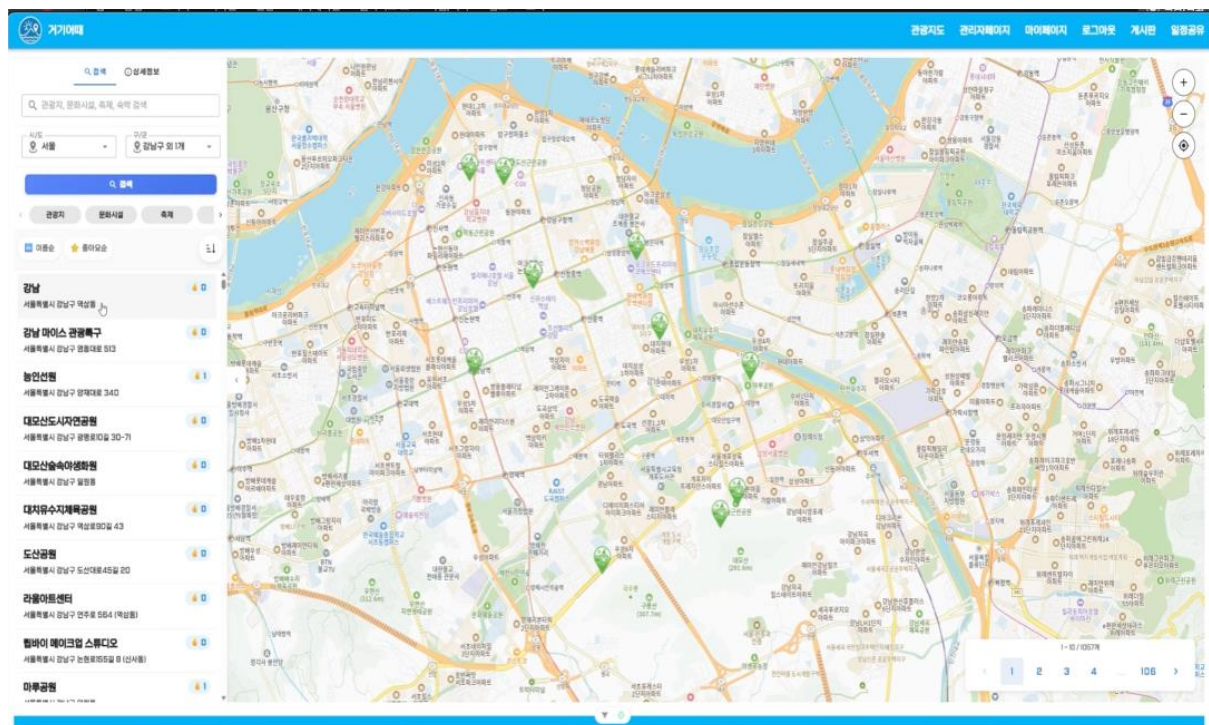
AI Spring AI , OPENAI - API

Infra Docker, Docker-compose, Amazon S3, Ubuntu EC2

Preview – (1)



메인페이지(오늘의 추천 관광지 및 오늘의 날씨 정보)

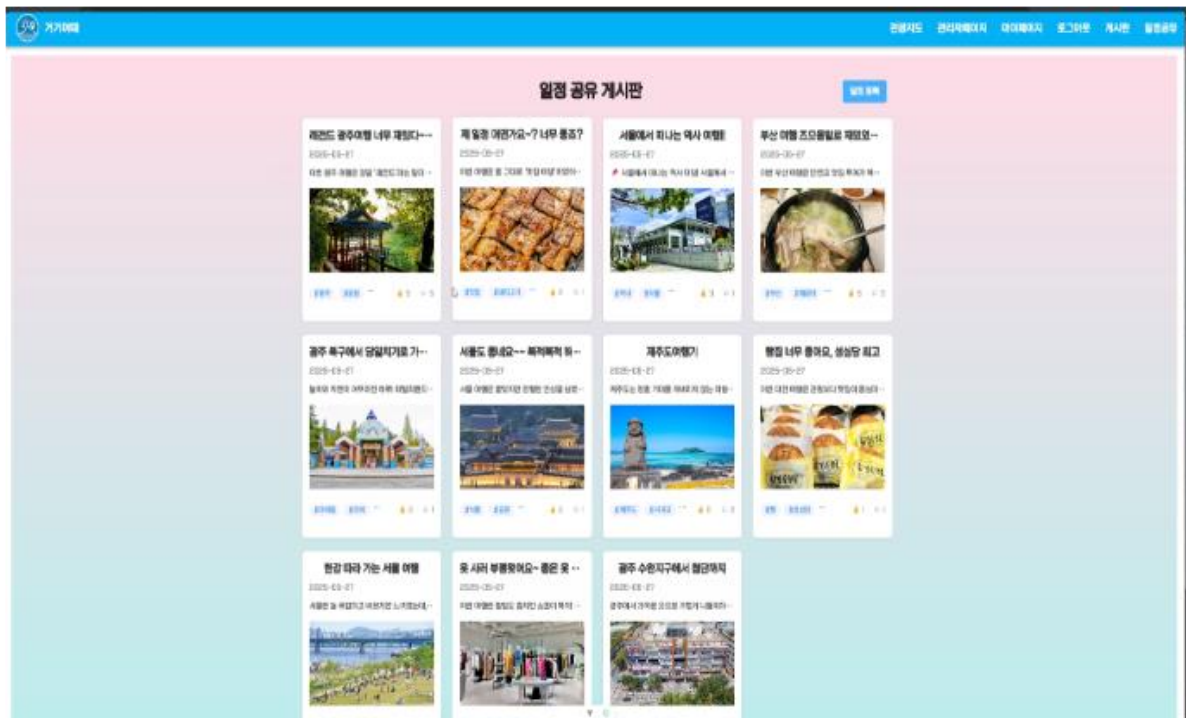


관광지 검색 및 AI 관광지 추천

Preview – (2)



여행 일정 상세페이지(최단거리 확인 및 게시판 기능)



일정 공유 게시판

담당 업무

추천 일정 최단거리 기능 및 일정 게시판 관련 디자인

! Kakao API 사용한 일정 최단거리 기능 구현 및 일정 게시판 CRUD

- Spring Boot와 MyBatis를 활용하여 일정 게시판 CRUD 기능 구현
- Kakao API를 활용하여 일정 최단거리 백엔드 및 프론트엔드 기능 구현
- 일정 게시판 관련 프론트엔드 및 디자인 구현

개별 여행지 추천 및 주변 여행지 추천 기능

! Spring AI 및 Redis를 활용하여 사용자 데이터 기반 서비스 개발

- 즐겨찾기에 등록된 장소를 기준으로 Redis에 벡터데이터로 저장 후 Spring AI를 통해서 Redis에 하루동안 5가지 장소를 추천해주는 기능 개발
- 장소 검색을 통해서 AI 주변 관광지 추천 기능 개발

Troubleshooting

KMP 알고리즘을 적용하여 검색 속도 향상

? 검색어를 입력하여 포함된 검색어를 찾는데 $O(n^2)$ 시간 소요

✓ KMP 알고리즘을 사용하여 $O(n^2) \rightarrow O(N+M)$ 으로 향상

- 단순 검색 기능이 아니라 KMP 문자열 매칭 알고리즘을 도입하여 기존 검색 기능을 부분 일치 기반 장소 검색 시스템으로 개선하고 검색 효율성을 극대화
- 시간복잡도를 $O(n^2) \rightarrow O(N+M)$ 로 향상함

다락시장 다락

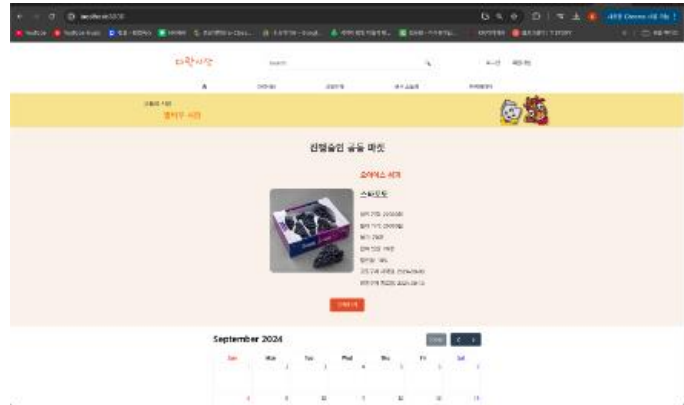
전통시장 공동구매 플랫폼

2024.08.06~2024.08.26 (3주/ 5인)

참고 링크



<https://github.com/lwy-0418/2024-Oasis-Darak>



역할

Backend(100%)

팀원 구성

Frontend(2) | **Backend** 2 | Plan 1

특이사항

오아시스 특별상(오아시스 해커톤)

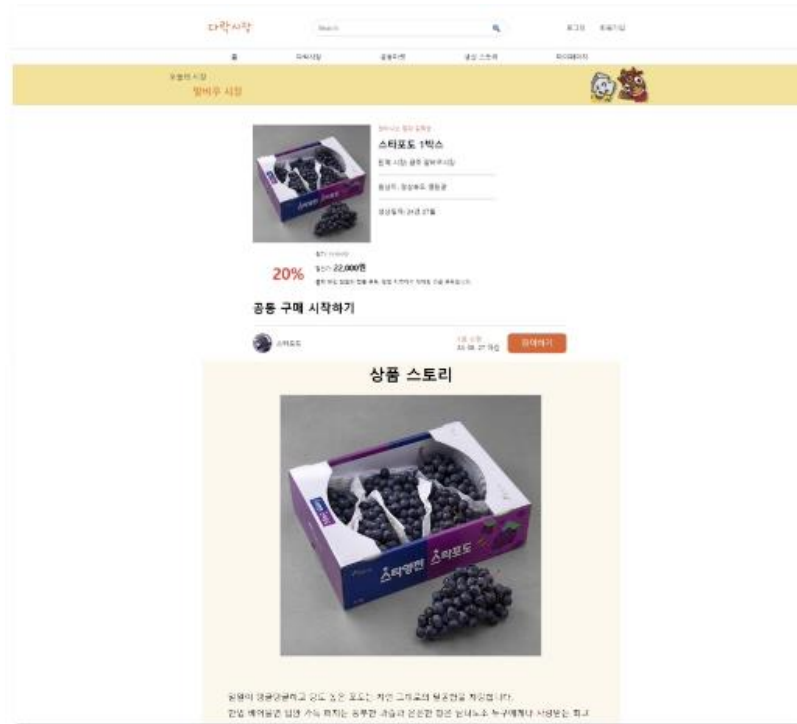
개발환경

Frontend React

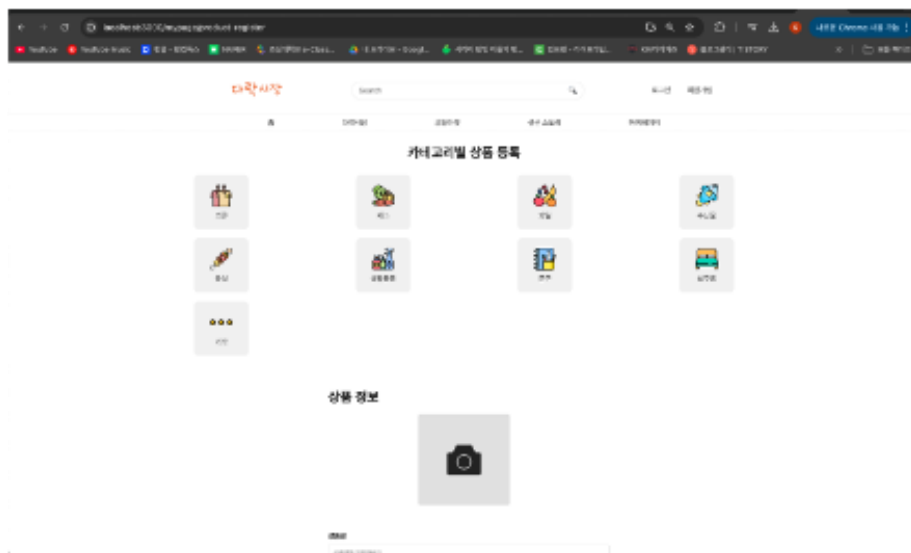
Backend Java17, Spring Boot, JPA , toss payment API

DataBase MySql

Preview – (1)

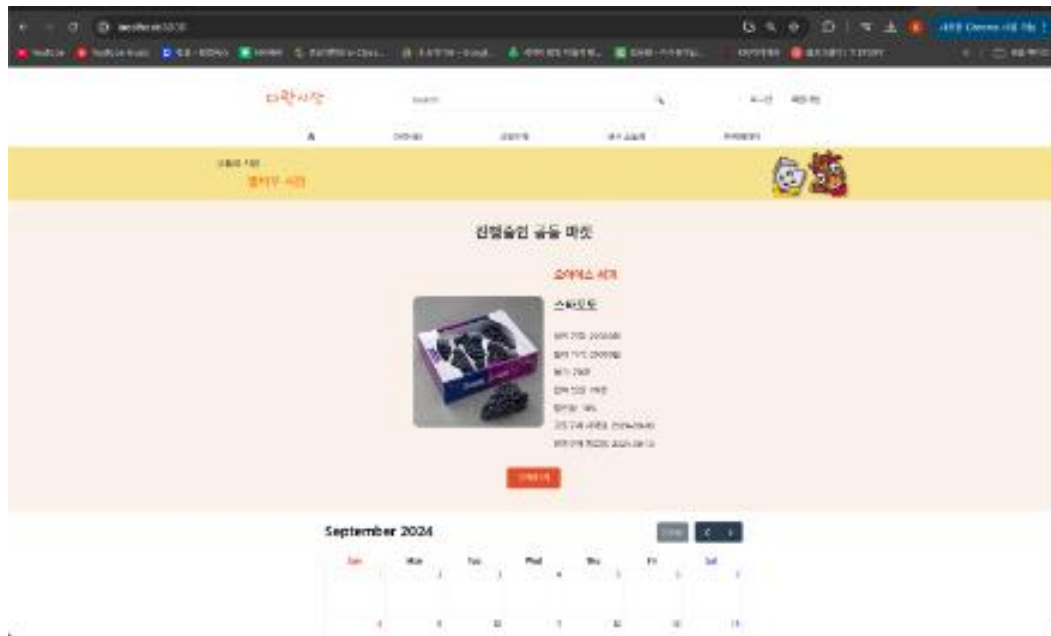


공동 구매 신청 페이지



상품 등록 페이지

Preview – (2)



메인 페이지

04. Solution Business Model



OASIS HACKATON

비즈니스 모델 구조도

담당 업무

Toss Payment Api 및 제품 API 개발 담당

! Toss Payment API 및 제품 API 구조 설계 및 개발

- Spring Boot와 JPA를 활용하여 제품 등록, 수정, 조회 기능 구현
- Toss Payment API를 활용하여 공동 구매 결제 기능 구현

Troubleshooting

전통시장 문제점을 찾아 직접 상인과의 인터뷰 진행

🔗 우리가 느끼는 것보다 직접 상인들이 느끼는 문제점의 인지 필요

☑ 혼자서 직접 양동시장에 찾아가 상인분들과의 인터뷰를 통해 문제점을 파악함

- 온라인 플랫폼의 부재로 인해서 기타 대형매장에 비해 홍보 및 배달의 인프라가 갖춰지지 않은 문제점을 알 수 있었고 대형 매장의 할인 판매로 인해 가격 경쟁력을 가질 수 없는 문제점을 알게 됨
- 공동구매라는 방식을 통해서 가격경쟁력 및 홍보 효과를 얻었고 이를 통해 테스트 시행시에 상인들의 이익은 30% 향상되었고 하루 일평균 방문자수가 약 100명에서 200명으로 늘어나는 효과를 받음